

HANDS-ON 5 [Intermediate]

Flask with SQLAlchemy ORM & Database Integration

Topics Covered:

Flask-SQLAlchemy Setup✓

Model Definition✓

Database Migrations with Flask-Migrate✓

ORM CRUD Operations✓

Relationship Queries✓

Connection Pooling✓

You will integrate SQLAlchemy into the Flask Course Management API, define models, run migrations, and perform all database operations through the ORM — connecting the API layer built in Hands-On 4 to a real database.

Task 1: Define SQLAlchemy Models and Migrations

Goal: Set up Flask-SQLAlchemy and define models matching the course management schema.

Steps:

47.

In `app.py`, initialise Flask-SQLAlchemy: `db = SQLAlchemy()`. In `create_app()`, call `db.init_app(app)`.

48.

In `courses/models.py`, define `Department`, `Course`, `Student`, and `Enrollment` SQLAlchemy models using `db.Model` as the base. Mirror the same schema used in the Django hands-on.

49.

Define relationships: `Course` has a `db.relationship('Department', back_populates='courses')`. `Enrollment` has relationships to both `Student` and `Course`.

50.

Initialise Flask-Migrate: `migrate = Migrate(app, db)`. Run `flask db init`, `flask db migrate -m 'initial schema'`, `flask db upgrade`. Verify tables are created.

51.

Use the Flask shell (`flask shell`) to insert 2 departments and 3 courses via the ORM. Commit using `db.session.commit()`.

Hint:

-

Always call `db.session.commit()` to persist changes. `db.session.add(obj)` stages the object but does not save it yet.

-

`flask db migrate` only detects column additions/deletions reliably — relationship changes may need manual migration edits.

Expected Outcome: `flask db upgrade` creates all tables. Flask shell inserts work. `db.session.query(Course).all()` returns the inserted courses.

Task 2: Connect ORM to API Routes

Goal: Replace in-memory data in the Flask routes with real database queries.

Steps:

52.

Update the GET `/api/courses/` route to query the database: `Course.query.all()` (or `db.session.execute(db.select(Course)).scalars()`). Serialise results to a list of dicts using a `to_dict()` method on the model.

53.

Add a `to_dict()` method to each model that returns a dictionary of its fields — this is Flask's equivalent of DRF serializers.

54.

Update POST `/api/courses/` to create a `Course` object, add it to the session, and commit. Return the new course as JSON with status 201.

55.

Update GET `<int:id>/`, PUT, and DELETE to use `Course.query.get_or_404(id)` — this automatically returns a 404 JSON response if the course is not found.

56.

Add a route GET `/api/courses/<int:id>/students/` that uses a JOIN query to return all students enrolled in the specified course.

Hint:

-

`to_dict()` on models is a common Flask pattern. For larger projects, consider `marshmallow` or `flask-marshmallow` for serialization.

Page | For Digital Nurture 5.0 Participants Only

Digital Nurture 5.0 | Python Backend Frameworks | Hands-On Exercise Book

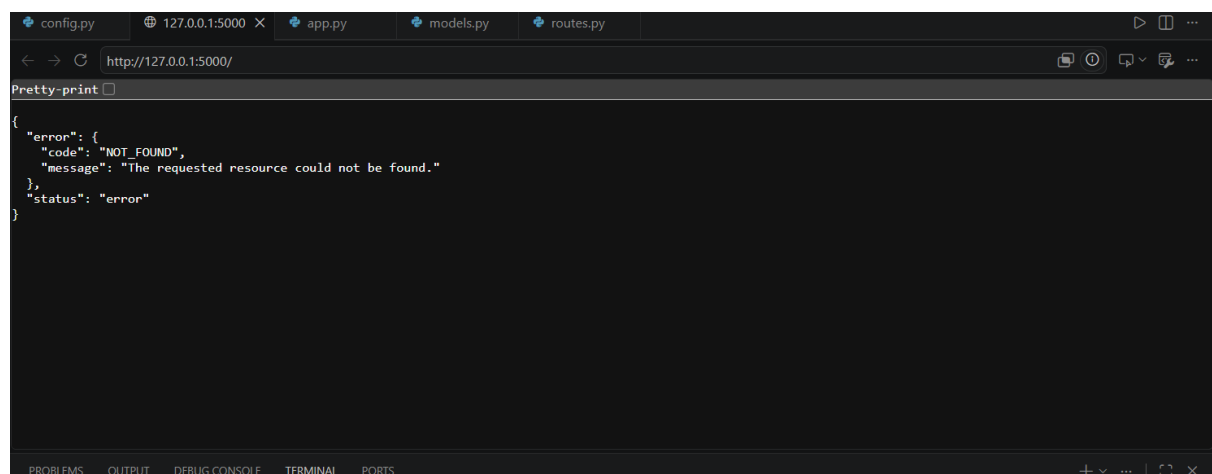
-

`Course.query.get_or_404(id)` is the Flask-SQLAlchemy shortcut that combines a primary key lookup with a 404 abort in one line.

Expected Outcome: All CRUD endpoints read from and write to the database.

The `/students/` route returns the correct enrolled students via a JOIN.

Outcome:



The screenshot shows a web browser window with the address bar displaying `http://127.0.0.1:5000/`. The browser's developer tools are open, showing a JSON error response in the console. The response is:

```
{
  "error": {
    "code": "NOT_FOUND",
    "message": "The requested resource could not be found."
  },
  "status": "error"
}
```

The browser's tab bar shows several files: `config.py`, `127.0.0.1:5000`, `app.py`, `models.py`, and `routes.py`. The bottom status bar of the browser shows tabs for `PROBLEMS`, `OUTPUT`, `DEBUG CONSOLE`, `TERMINAL`, and `PORTS`.

config.pyapp.py127.0.0.1:5000/api/courses/ Xmodels.pyroutes.py

http://127.0.0.1:5000/api/courses/

Pretty-print

```
{
  "data": [
    {
      "code": "CS301",
      "credits": 4,
      "department_id": 1,
      "id": 1,
      "name": "Python Web Frameworks"
    },
    {
      "code": "CS302",
      "credits": 3,
      "department_id": 1,
      "id": 2,
      "name": "Relational Databases"
    },
    {
      "code": "IT401",
      "credits": 3,
      "department_id": 2,
      "id": 3,
      "name": "Cloud Solutions"
    }
  ],
  "status": "success"
}
```

PROBLEMSOUTPUTDEBUG CONSOLETERMINALPORTS

```
(.venv) PS C:\Users\admin\Downloads\Digital-Nurture-PythonFSE-main\Digital-Nurture-PythonFSE-main\Python FSE\Deepskilling\Python Backend Hands-on\project\flask_coursemanager\flask_coursemanager> python app.py
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 747-355-123
127.0.0.1 - - [01/Jul/2026 10:23:18] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [01/Jul/2026 10:23:24] "GET /api/courses/ HTTP/1.1" 200 -

```

powersh...python...

Go Live